



---

DDA5001 · Homework 1

Due: 23:59, Sep 30.

**Instructions:**

- Homework problems must be clearly answered to receive full credit. Complete sentences that establish a clear logical progress are highly recommended.
- Submit your answer paper in Blackboard. Prepare your answers in a **PDF** file and saving all your code as **.py** files. Combine the PDF and code files into a single .zip file for submission. The file name should be in the format **last name-first name-hw1**; e.g., **San-Zhang-hw1**.
- You may submit your homework multiple times before the deadline. Only your **latest submission will be graded**.
- The homework must be written in English.
- Late submission will not be graded.
- Each student **must not copy** homework solutions from another student or from any other source. You are encouraged to discuss with others. However, you must write down the answer using your understanding (after discussion) and words.

---

**Problem 1 (12 points). Concepts and Fundamental Knowledge**

- (a) (4 points) Concisely state the difference between supervised learning and unsupervised learning.
- (b) (4 points) Which one of the following statements is true?
- 1) Regression is used to fit categorical labels.
  - 2) Least squares must have infinitely many solutions if the number of data points is smaller than feature dimension.
  - 3) The perceptron always converges to a unique linear classifier for a given training dataset.
  - 4) Least squares is a maximum likelihood estimator regardless of the noise distribution.
- (c) (4 points) Let the data matrix  $\mathbf{X} \in \mathbb{R}^{n \times d}$  be a full column rank matrix. Explain why  $\mathbf{X}^\top \mathbf{X}$  is positive definite (PD). (Hint: Use the definition of linear independence)

Since a PD matrix must be invertible as well, this clarifies why we can take matrix inverse to obtain the unique solution of least squares in case I (where  $\mathbf{X}$  has full column rank). From an optimization perspective, this PD property also reveals that the least squares in case I is strongly convex.

**Problem 2 (13 points). Least Squares Without Full Column Rank**

Consider the least squares problem

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2,$$

where  $\mathbf{X} \in \mathbb{R}^{n \times d}$ ,  $\boldsymbol{\theta} \in \mathbb{R}^d$ ,  $\mathbf{y} \in \mathbb{R}^n$ .

- (a) (8 points) When  $n < d$ , the problem yields infinite many solutions. Assume that  $\text{rank}(\mathbf{X}) = n$ , derive the expression of the infinitely many solutions based on singular value decomposition (SVD). In addition, what is the optimal function value?

Note that the SVD is a notion from linear algebra, and the SVD of the matrix  $\mathbf{X}$  can be written as

$$\mathbf{X} = \underbrace{\mathbf{V}}_{\in \mathbb{R}^{n \times n}} \underbrace{\begin{bmatrix} \boldsymbol{\Sigma}_1 & \mathbf{0} \end{bmatrix}}_{\in \mathbb{R}^{n \times d}} \underbrace{\begin{bmatrix} \mathbf{U}_1^\top \\ \mathbf{U}_2^\top \end{bmatrix}}_{\in \mathbb{R}^{d \times d}} = \mathbf{V}\boldsymbol{\Sigma}_1\mathbf{U}_1^\top.$$

where  $\mathbf{V} \in \mathbb{R}^{n \times n}$  is an orthogonal matrix such that  $\mathbf{V}^\top \mathbf{V} = \mathbf{V}\mathbf{V}^\top = \mathbf{I}$ ,  $\boldsymbol{\Sigma}_1 \in \mathbb{R}^{n \times n}$  is a diagonal matrix taking the form

$$\boldsymbol{\Sigma}_1(i, j) = \begin{cases} \sigma_i, & i = j \\ 0, & i \neq j \end{cases} \quad \text{with} \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n > 0,$$

$\mathbf{U}_1^\top \in \mathbb{R}^{n \times d}$  is an semi-orthogonal matrix satisfying  $\mathbf{U}_1^\top \mathbf{U}_1 = \mathbf{I}$  (but  $\mathbf{U}_1 \mathbf{U}_1^\top \neq \mathbf{I}$ ), and  $\mathbf{U}_2^\top \in \mathbb{R}^{(d-n) \times d}$  is an semi-orthogonal matrix satisfying  $\mathbf{U}_2^\top \mathbf{U}_2 = \mathbf{I}$  (but  $\mathbf{U}_2 \mathbf{U}_2^\top \neq \mathbf{I}$ ).

(Hint: Let  $\mathbf{A} := \mathbf{V}\boldsymbol{\Sigma}_1$  which is square matrix of full rank,  $\mathbf{z} = \mathbf{U}_1^\top \boldsymbol{\theta}$ , solve  $\min_{\mathbf{z}} \|\mathbf{A}\mathbf{z} - \mathbf{y}\|_2^2$  first, then solve  $\mathbf{U}_1^\top \boldsymbol{\theta} = \mathbf{z}$  by considering that  $\mathbf{U}_1^\top \mathbf{U}_2 = \mathbf{0}$ .)

- (b) (5 points) Later in the overfitting section, we will study that we can also add a regularizer to solve the issue of infinitely many solutions, resulting in the following problem:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2,$$

where  $\lambda$  is a positive constant. Derive the unique solution of this problem.

### Problem 3 (23 points). Robust Linear Regression with a Coding Agent

**AI collaboration for Problems 3 and 5.** For these two problems, you are encouraged to use a coding agent to help with derivation, implementation, debugging, and analysis. This permission takes precedence over the general restriction on using other sources insofar as it concerns AI assistance on these two problems. You remain responsible for understanding and explaining your submission. Copying another student's work or fabricating conversations or experimental results is not permitted.

For each problem, record one initial, testable expectation before running your verification, use an agent to help implement the task, and bring a concrete verification result back to the agent for a focused follow-up. Explain your final decision using mathematical or numerical evidence. An initially correct agent solution can receive full credit. There is no requirement to find an AI error or to produce a minimum number of chat turns.

Use `code_source/p3/p3.py`. Data loading, plotting, and output scaffolding are provided. Complete the marked mathematical and verification functions, with agent assistance. Submit your completed code and numerical outputs together with your answer PDF. Include one AI collaboration case for this problem, at most one page, as specified in part (e).

Suppose the linear regression model is

$$\mathbf{y} = \mathbf{X}\boldsymbol{\theta}^* + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \quad \sigma > 0,$$

where  $\mathbf{X} \in \mathbb{R}^{n \times d}$  has full column rank. Under this noise assumption, the maximum likelihood estimator is the least-squares solution

$$\hat{\boldsymbol{\theta}}_{LS} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

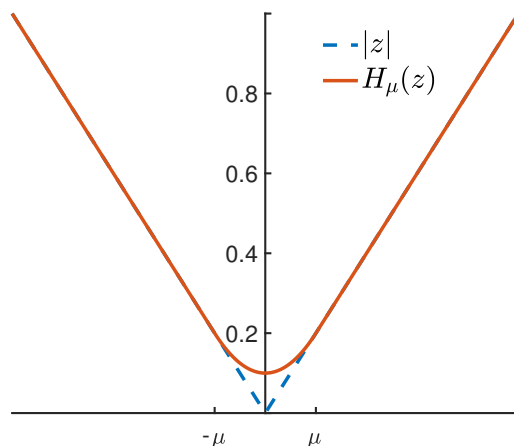
- (a) **(3 points) Laplace noise and the loss function.** Instead suppose that the errors are i.i.d. Laplace with density

$$p(\varepsilon_i) = \frac{1}{2b} \exp(-|\varepsilon_i|/b), \quad b > 0.$$

Derive the optimization problem for estimating  $\boldsymbol{\theta}^*$  by maximum likelihood. Show the negative log-likelihood and identify the terms that do not depend on  $\boldsymbol{\theta}$ .

- (b) **(3 points) Huber smoothing and its gradient.** For  $\mu > 0$ , define the following smoothing of the absolute value:

$$h_\mu(z) = \begin{cases} |z|, & |z| \geq \mu, \\ \frac{z^2}{2\mu} + \frac{\mu}{2}, & |z| < \mu, \end{cases} \quad \mathcal{L}(\theta) = \sum_{i=1}^n h_\mu((\mathbf{X}\theta - \mathbf{y})_i).$$



**Geometric intuition.** The blue dashed curve is  $|z|$ , and the orange curve is its smoothing  $h_\mu(z)$ . On  $(-\mu, \mu)$ , a quadratic arc replaces the sharp corner of  $|z|$ ; outside this interval, the two curves coincide. This makes the loss continuously differentiable for gradient-based optimization, while retaining linear growth for large residuals, so outliers have less influence than under squared loss.

Derive  $h'_\mu(z)$  and  $\nabla \mathcal{L}(\theta)$ , with the dimensions of the vectors in your expression. Use this exact definition of  $h_\mu$  and a *sum* over samples in the implementation. A library function named “Huber” may use a different scaling or additive constant.

- (c) **(6 points) Agent-assisted implementation and baseline experiment.** The supplied data follow

$$\mathbf{y} = \mathbf{X}\theta^* + \epsilon_1 + \epsilon_2,$$

where  $\epsilon_1$  is Gaussian noise and  $\epsilon_2$  contains sparse outliers. Load the supplied  $X$ ,  $y$ , and  $\theta^*$  from `data/`;  $n = 1000$  and  $d = 50$ .

- (1) Compute the least-squares estimate and report  $\|\hat{\theta}_{LS} - \theta^*\|_2$ . Use a numerical least-squares solver rather than explicitly forming a matrix inverse.
- (2) Implement

$$\theta_{k+1} = \theta_k - \alpha \nabla \mathcal{L}(\theta_k), \quad k = 0, \dots, T-1,$$

using  $\mu = 10^{-5}$ ,  $\alpha = 0.001$ , and  $T = 1000$ . Use the provided initialization:

```
rng = np.random.default_rng(0)
theta0 = rng.standard_normal(d)
```

Record  $T + 1$  states, including  $k = 0$  and  $k = T$ .

- (3) Plot  $\|\theta_k - \theta^*\|_2$  versus  $k$ , with the least-squares error as a horizontal reference. Save the objective values and parameter errors as a CSV file. Report the seed and command needed to reproduce the outputs. Use  $\theta^*$  only for evaluation, never to compute an update or choose an iterate.
- (d) **(5 points) Verify the agent’s implementation.** Complete the verification TODOs and submit a small table of numerical results.
- (1) Check the scalar derivative at residuals  $(-2, -0.5, 0, 0.5, 2)\mu$ , covering both outer branches and the inner branch. Then use the supplied small matrix to compare a directional derivative from your gradient with a finite-difference approximation at several step sizes. Run this check with  $\mu = 1$  and with  $\mu = 10^{-5}$ . Explain how it is related to the first-order Taylor expansion from slides 1, and why an excessively small or large finite-difference step can be misleading. A table of “passed” labels without the compared values and an explanation is insufficient.
  - (2) Verify on the same small example that replacing the sum loss by the mean loss divides the gradient by the number of samples. Explain how the learning rate would have to change to obtain the same update. Check whether the agent’s baseline code implements the sum or the mean.

The small example and step-size scaffolding are supplied so that this check requires no additional optimization theory.

- (e) **(6 points) AI collaboration case, at most one page.** Document one interaction connected to parts (b)–(d). You can consider to include: (i) your initial expectation and the issue you wanted to check; (ii) the tool/model, your actual prompt, and a short relevant excerpt of its response; (iii) the numerical evidence you brought back to the agent and a relevant excerpt of the follow-up; and (iv) your decision to revise or retain the result, supported by the derivative formula, verification table, or code. Reference your submitted files rather than duplicating all outputs. Retain the relevant original conversation for possible review. Do not manufacture a failure if the first implementation was correct.

**Problem 4 (24 points). Convergence of The Perceptron for Linearly Separable Data**

In lecture, we studied that one iteration of perceptron algorithm will move forward to the currently wrongly classified data. Though such an observation does provide some intuition that perceptron will eventually converge to a linear classifier that correctly separate all the (linearly separable) training data, it is not rigorously justified.

Let us first review the algorithm. The perceptron applies to the linear model

$$f_{\theta}(\mathbf{x}) = \theta^{\top} \mathbf{x}.$$

Here  $\mathbf{x}$  includes a constant 1 as a bias coordinate, and  $y_i \in \{+1, -1\}$ . The diagrams show the original two-dimensional feature space. Define  $\text{sign}(z) = +1$  for  $z \geq 0$  and  $\text{sign}(z) = -1$  otherwise, as in Problem 5.

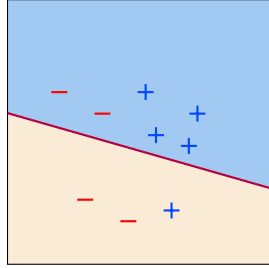
At the  $k$ -th mistake update, the current parameter is  $\theta_{k-1}$ . Choose any misclassified training sample, denoted by  $(\mathbf{x}_{k-1}, y_{k-1})$  for this update, so that

$$\text{sign}(\theta_{k-1}^{\top} \mathbf{x}_{k-1}) \neq y_{k-1}.$$

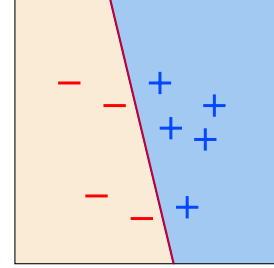
Then, perceptron updates as

$$\theta_k = \theta_{k-1} + y_{k-1} \mathbf{x}_{k-1} \quad \forall k = 1, 2, \dots \quad (1)$$

In this question, you will be guided to show that perceptron terminates after finitely many mistake updates on strictly linearly separable data; see Fig. 1. The count  $k$  refers to parameter updates, not full passes through the dataset (epochs).



(a) Perceptron at the  $k$ -th iteration, not all data are correctly classified



(b) Learned perceptron model after at most  $\bar{k}$  iterations, all data are correctly classified

Figure 1: Illustration of the convergence progress of the perceptron learning algorithm.

Assume the data are strictly linearly separable: choose  $\theta^*$  such that  $y_i \theta^{*\top} \mathbf{x}_i > 0$  for every training sample  $i$ , as illustrated in Fig. 1b. For simplicity, initialize the perceptron at  $\theta_0 = \mathbf{0}$ .

The full proof procedure is divided into 5 steps.

- (a) (3 points) Let  $\rho = \min_{1 \leq i \leq n} y_i (\theta^{*\top} \mathbf{x}_i)$ . Show that  $\rho > 0$ .
- (b) (6 points) Show that  $\theta_k^{\top} \theta^* \geq \theta_{k-1}^{\top} \theta^* + \rho$ , and hence conclude that  $\theta_k^{\top} \theta^* \geq k\rho$ .  
(Hint: Use the update (1).)

(c) (6 points) Show that  $\|\boldsymbol{\theta}_k\|^2 \leq \|\boldsymbol{\theta}_{k-1}\|^2 + \|\mathbf{x}_{k-1}\|^2$ .

(Hint: Use the fact that  $\mathbf{x}_{k-1}$  is misclassified by  $\boldsymbol{\theta}_{k-1}$ .)

(d) (3 points) Show that  $\|\boldsymbol{\theta}_k\|^2 \leq kR^2$ , where  $R = \max_{1 \leq i \leq n} \|\mathbf{x}_i\|$ .

(e) (6 points) For  $k \geq 1$  after  $k$  mistake updates, use steps 2 and 4 to show that

$$\frac{\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^*}{\|\boldsymbol{\theta}_k\|} \geq \sqrt{k} \frac{\rho}{R}$$

and hence show that the perceptron learning algorithm must stop within at most

$$\bar{k} \leq \frac{R^2 \|\boldsymbol{\theta}^*\|^2}{\rho^2}$$

mistake updates.

(Hint: Use the fact  $\frac{\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^*}{\|\boldsymbol{\theta}_k\| \|\boldsymbol{\theta}^*\|} \leq 1$ . Why?)

Perceptron often terminates before this bound. A numerical bound requires a known separating vector and margin. For nonseparable data, the strict-separation assumption fails, so this guarantee does not apply.

### Problem 5 (28 points). Perceptron and Pocket with a Coding Agent

The AI collaboration permission and workflow stated in Problem 3 apply here. Use `code_source/p5/p5.py`. Submit one additional AI collaboration case of at most one page. The two cases together must not exceed two pages.

For nonseparable data, ordinary perceptron need not terminate and its training error can fluctuate. Pocket keeps the parameter with the smallest training error seen so far. Its stored training error is therefore *nonincreasing*; there is no analogous guarantee for its test error.

Use the supplied US Post Office digit data: `train_data.mat` and `test_data.mat`. The full datasets contain 7291 training images and 2007 test images of size  $16 \times 16$ . Extract digits 1 and 6, assigning label +1 to digit 1 and -1 to digit 6. The supplied feature code computes mean pixel intensity and left-right asymmetry. Augment these with a bias coordinate:

$$\tilde{x}_i = (1, \text{intensity}_i, \text{asymmetry}_i)^\top, \quad \theta \in \mathbb{R}^3.$$

Predict +1 when  $\theta^\top \tilde{x}_i \geq 0$  and -1 otherwise. This fixes the zero-score convention; do not subtract an additional threshold from the score. Count misclassification using exactly this prediction rule.

At a mistake update, choose a misclassified *training* sample uniformly at random and set

$$\theta_{k+1} = \theta_k + y_i \tilde{x}_i.$$

Initialize  $\theta_0 = \hat{\theta}_0 = 0$ , use random seed 0, and make at most  $T = 2000$  mistake updates. Stop early if no training sample is misclassified. Update the pocket only when the new training error is strictly smaller, and store a copy of the parameter. Continue the next perceptron step from  $\theta_{k+1}$ , not from the pocket.

To make the comparison controlled, record the current perceptron parameter and the pocket parameter along the *same* update trajectory. Store the initial state and every subsequent state. Compute test errors from these saved states only after training has finished; test labels must not influence updates, pocket selection, or the stopping rule.

- (a) **(7 points) Agent-assisted implementation.** Complete the perceptron-step and pocket-update TODOs. The supplied code handles loading, features, random sampling, trajectory collection, and plots. Explain the difference between the current parameter and the pocket parameter, and point to the code that implements each update. Record the command and seed needed to reproduce your run.
- (b) **(6 points) Error curves and interpretation.** Plot both training and test errors of the current perceptron and of the pocket versus the number of mistake updates. Save the numerical trajectory as a CSV file. Explain which curve has a monotonicity guarantee and why. Describe the actual behavior of your test curves without claiming that pocket must improve test error at every step or outperform the current perceptron on every run.
- (c) **(4 points) Features and decision boundaries.** Use the provided plotting code to show 500 training images from each of digits 1 and 6 in feature space, or all available images if a class contains fewer than 500. Overlay the final current-perceptron and pocket boundaries on the same axes. Label both axes and both boundaries. Explain the role of the bias coordinate. If a boundary is vertical or the normal vector is zero, handle and explain that case instead of dividing by zero.

- (d) **(5 points) Audit the pocket using a small trace.** Before running the small example below, write one or two sentences predicting which error sequence should be nonincreasing and whether this property should extend to test error.

$$\tilde{X}_{toy} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}, \quad y_{toy} = [1 \quad -1 \quad 1 \quad -1]^\top.$$

The first two samples have identical inputs and opposite labels, so these data are nonseparable. Use the same prediction rule and zero initialization, seed 0, and six mistake updates.

- (1) Complete the history-audit TODO and save a seven-row trace (initialization plus six updates) showing the actual current and pocket weights and their training errors. Select one update from this trace and verify its error calculation and pocket decision explicitly.
- (2) Check numerically that the pocket error equals the running minimum of the current training errors, including initialization:

$$E_{in}(\hat{\theta}_k) = \min_{0 \leq j \leq k} E_{in}(\theta_j).$$

Explain why storing the current parameter without a copy can invalidate the intended pocket state when the current parameter is modified in place.

- (3) State whether a temporary rise in pocket test error, by itself, shows that the implementation is wrong. Support your answer using the selection rule.

Do not edit a trace to force the expected property. If a check fails, retain the initial evidence and explain how you diagnosed it.

- (e) **(6 points) AI collaboration case, at most one page.** Bring one concrete result from parts (b)–(d) back to your agent and ask a focused question about the implementation or the algorithm’s guarantee. You can consider to include your initial prediction, tool/model, actual prompt and relevant response excerpt, numerical evidence, a relevant follow-up excerpt, and your decision with justification. For example, you might examine whether a plotted curve uses current or pocket weights, why a test curve rises, or whether saved arrays change after a later update. Use an issue relevant to your actual run. A verified correct implementation needs no artificial bug or forced revision.